

30-Day AI + Backend (FastAPI) Roadmap — Internship-Ready

Starting point: You already do basic CRUD in FastAPI. Good — that's the 20% base. This roadmap builds the *next* 20% of skills that account for 80% of what's actually screened for in 2026 AI/backend internship postings (LangChain + RAG, Docker, deployment, LLM APIs, vector DBs, auth).

Rule of the month: Every week ends with something *deployed and demoable* — not just code on your laptop. Recruiters skim resumes in seconds but spend real time on live demos and clean GitHub repos.

The 80/20 Skill Map (what actually gets you shortlisted)

High-leverage 20% (learn this)	Low-leverage 80% (skip/skim for now)
FastAPI: async endpoints, Pydantic models, dependency injection, JWT auth	GraphQL, gRPC, niche frameworks
PostgreSQL + SQLAlchemy/SQLModel, Alembic migrations	Exotic databases, manual query tuning
Docker (Dockerfile + docker-compose)	Kubernetes (mention only, don't deep-dive yet)
LLM APIs (OpenAI/Anthropic) — function/tool calling, structured output, streaming	Training your own LLM from scratch
RAG: embeddings, chunking, vector DB (Chroma or pgvector), retrieval + re-ranking	Fine-tuning (nice-to-have, not required for internship)
LangChain or LangGraph basics (just enough to orchestrate)	Mastering every LangChain module
Git/GitHub workflow, clean READMEs, one live deployment (Render/Railway/Fly.io)	CI/CD pipelines (basic GitHub Actions is a bonus, not core)

Week 1 — Harden your FastAPI backend fundamentals

Goal by end of week: A production-shaped FastAPI backend (not tutorial-CRUD) with real auth, a real database, and validation — pushed to GitHub.

- Day 1-2: Rebuild your CRUD app using **SQLModel or SQLAlchemy + PostgreSQL** (not SQLite). Add proper Pydantic request/response schemas (separate from DB models).

- Day 3: Add **JWT authentication** (login, signup, protected routes) using `python-jose` or `fastapi-users`.
 - Day 4: Add **async endpoints** properly (`async def`), async DB sessions with `asyncpg`). Learn *why* async matters for I/O-bound AI calls — this connects directly to Week 2.
 - Day 5: Add **Alembic migrations**, environment variables (`.env` + `pydantic-settings`), proper error handling (custom exception handlers).
 - Day 6: Write a clean `README.md` (setup, architecture diagram, endpoints table). Push to GitHub with meaningful commits (not one giant commit).
 - Day 7: **Dockerize it**. Write a Dockerfile + docker-compose (FastAPI + Postgres). Deploy to Render or Railway (free tier). This is your Week 1 checkpoint — a live URL you can share.
-

Week 2 — Bring AI into the backend (LLM integration)

Goal by end of week: Your FastAPI app can call an LLM, handle streaming responses, and return structured output.

- Day 8-9: Get an OpenAI or Anthropic API key. Build a `/chat` endpoint that calls the LLM and returns a response through FastAPI.
 - Day 10: Learn **streaming responses** in FastAPI (`StreamingResponse`) so the LLM's output streams token-by-token to the client — this is a real production pattern interviewers ask about.
 - Day 11: Learn **structured output / function calling** — force the LLM to return JSON matching a Pydantic schema. This is one of the highest-signal skills right now.
 - Day 12: Add **rate limiting** and basic cost/token tracking on your AI endpoints (shows production awareness, not just "I called an API").
 - Day 13-14: Combine Week 1 + Week 2: an authenticated user can chat, and conversation history is stored in Postgres. Deploy this update.
-

Week 3 — Build a real RAG pipeline (the highest-demand skill in 2026)

Goal by end of week: A working Retrieval-Augmented Generation system — the single most requested AI engineering skill right now.

- Day 15: Learn **chunking + embeddings** conceptually (why chunk size matters, overlap, embedding models like `text-embedding-3-small`).
- Day 16: Set up a **vector database** — use `pgvector` (since you already have Postgres — reuses your Week 1 setup) or Chroma for simplicity.
- Day 17-18: Build the pipeline: upload a PDF → chunk → embed → store in vector DB → retrieve relevant chunks on query → pass to LLM with context.
- Day 19: Add **LangChain** (or do it manually first, then swap in LangChain to see what it abstracts — this makes you actually understand it instead of copy-pasting).
- Day 20: Improve retrieval quality: try hybrid search or basic re-ranking. This is what separates a toy demo from something interviewers respect.

- Day 21: Wrap the whole RAG pipeline as FastAPI endpoints (`/upload`, `/query`), Dockerize, deploy. **This is your flagship project** — a "document Q&A over your own PDFs" app is exactly what current hiring guides call the foundational AI engineering portfolio piece.
-

Week 4 — Polish, one stretch project, and application sprint

Goal by end of week: Two live, documented projects + applications going out daily.

- Day 22: Add a minimal frontend to your RAG app (Streamlit or a simple HTML/JS page calling your API) — recruiters engage far more with things they can *click*, not just code.
 - Day 23: Add basic **observability**: log every LLM call (prompt, tokens, latency) to a table in Postgres. Mention this in your README — it signals LLMops awareness.
 - Day 24-25: **Stretch (optional, only if Week 1-3 is solid)**: Turn your RAG app into a simple agent — one tool-calling step (e.g., it can search the web or query a second data source before answering). Use LangGraph or just manual function-calling logic; don't over-engineer.
 - Day 26: Write final READMEs with architecture diagrams for both projects. Record a 2-minute Loom-style demo video for each — huge for applications and LinkedIn.
 - Day 27: Rewrite your resume around these two projects using this pattern: **what you built** → **stack used** → **one measurable detail** (latency, accuracy, scale) rather than a plain skills list.
 - Day 28-30: **Application sprint**. Apply to 5-10 internships/day targeting roles that mention FastAPI + LangChain/RAG/LLM (search terms: "GenAI intern," "AI backend intern," "LLM engineer intern"). Tailor your one-liner pitch per company. Post both projects on GitHub + LinkedIn with the demo videos.
-

Two Portfolio Deliverables You'll Have by Day 30

1. **Production-shaped FastAPI backend** — auth, Postgres, async, Docker, deployed.
2. **Deployed RAG document Q&A app** — the exact project current hiring guides describe as the standard AI engineering portfolio piece, with your own backend-engineering polish (auth, logging, streaming) on top of it — which most beginner portfolios lack.

Notes

- Don't chase every framework. LangChain is enough; skip LlamaIndex/CrewAI/AutoGen unless a specific job posting asks for it.
- Fine-tuning is *not* required for internships — RAG + solid API engineering carries far more weight at this stage.
- If a JD mentions "MCP" (Model Context Protocol) and you have spare time in Week 4, a 1-day read + toy MCP server integration is worth it — it's showing up as a differentiator in newer postings.